Optimizing US Railway Stations

Toben Feltman
Connor Fourman
Samyak Tuladhar
Chris Pageau

May 1, 2026

Abstract

This project will investigate how we can optimally place railway stations within the existing transportation network so that all areas in the continental United States are within a specified distance of a railway station. Using the METAL graph data, the goal is to minimize the number of additional stations needed to cover all the remaining regions.

1 Introduction

The logistic complexity of the continental United States presents a significant challenge for nationwide infrastructure planning. Especially within the rail sector. Ensuring comprehensive accessibility requires balancing the efficient use of resources with expansive coverage. This project aims to address this issue by investigating how to optimally place railroad stations throughout the country. Our objective is to ensure that every geographical region in the continental United States falls within a predefined maximum distance of a station.

To solve this problem, we utilized the METAL graph dataset, which provides a framework for modeling large-scale transit networks. The two algorithmic approaches we primarily used were Dijkstra's Algorithm and Greedy Set Cover.

We utilized Dijkstra's Algorithm in order to calculate the shortest paths and verify connectivity across the large set of nodes in the METAL dataset, ensuring that distance constraints are accurately measured between existing and potential stations. Greedy Set Cover was used to address the NP-hard nature of the problem. By iteratively selecting station locations that provide the maximum new coverage, we aim to minimize the total number of additional stations required. Since it isn't possible to find the perfect solution, this algorithm allows us to find one that is good enough, without sacrificing an unreasonable amount of time.

2 Computing Environment

```
java version "25.0.2" 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 25.0.2+10-LTS-69)
```

```
Java HotSpot(TM) 64-Bit Server VM (build 25.0.2+10-LTS-69, mixed
mode, sharing)
```

The computing environment is a MacBook Pro (14-inch, 2023) with an Apple M2 Pro, 16 GB of Micron LPDDR5 memory, and running macOS Sequoia Version 15.7.4. There are 10 cores, (6 performance and 4 efficiency).

3 Algorithm Analysis

Our Greedy Set Cover algorithm finds a path by prioritizing the shortest immediate connection at every step. While it operates with O(V + E) time complexity, it lacks "global" awareness, meaning it chooses the best local option without considering the total distance to the final destination. Therefore, unlike Dijkstra's algorithm, it does not guarantee the shortest overall path and is at risk of getting stuck in dead ends or cycles if the immediate shortest edges lead away from the goal. This highlights the trade-off between the speed of a greedy heuristic and the guaranteed optimality required for complex infrastructure planning.

3.1 Results

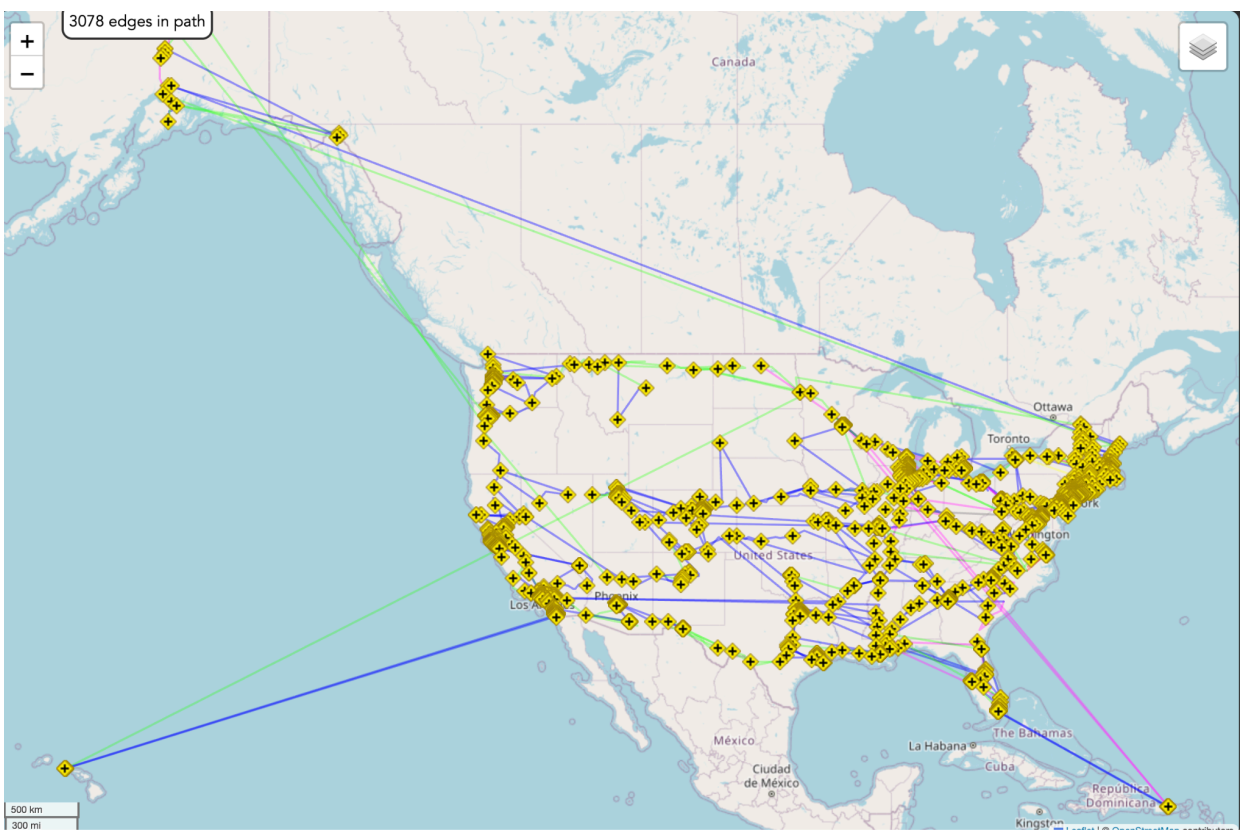

Figure 1: Results of Greedy Set Cover

4 Considerations for Implementations in Real Life

While the greedy set and the Dijkstra's algorithm have different advantages and compromises for the algorithms, this project is just a rudimentary test for the algorithms. Implementing this to the real world means taking into account a lot more factors which we haven't shown the limitations of this project. We wanted to show how the real world implementation of these algorithms into the real world would be much more complex and take into account many more factors.

4.1 Geography

The limitations of the both algorithms is that both of the algorithms make optimizations in either distance or 'coverage' in the sense of the greedy set cover. The real-world implementation would need to take into account some sense of geography. Forgetting the edge-cases of the edges spanning continental sides due to the disconnected nature of the US, we might need geography to take into account. The flat nature of the US makes it much more simple than other countries. But still in the geography in the Rockies, while the answers to both algorithms might make sense in their optimizations, one would need to consider geography for those edges to be realistically made between train stations.

4.2 Edge Capacity, i.e 'Traffic'

Another thing to consider is what type of road to use to join the railway trains. It is interesting to think that most of the new vertices may be in isolated populated centers so the edge weight might be minimal but it is still an interesting thing to consider. Also just maximizing the edge capacity might be the 'best' as it would theoretically take the least amount of time as it has more capacity but in real-life wouldn't really make sense. An extreme case would mean adding a maglev train track to a rural road which realistically doesn't make sense. But another more pertinent sense is to see if the railway stations can handle the additional traffic and capacity of the new edges. While 'traffic' in a railway station doesn't make sense, edge capacity can mean how many trips are done between two vertices in a day or a week, which would be interesting to see.

4.3 More Realistic Selection of Tracks

Another thing we have to consider is that we took all selections of tracks to show that yes, the US can be covered in tracks fairly well to make it a reasonable choice to be made. But what we've not considered is how many of the tracks are passenger tracks, hybrid tracks and what are cargo-oriented tracks. Sources say that while 140,000 miles of track exists in the US, only 21,000 miles of tracks are controlled by Amtrak showing the stark difference. Therefore, a real world implementation would need to begin with only passenger-only or hybrid tracks if the objective of the research is public transportation throughout the US through rail.

5 Conclusion

This project successfully showed how algorithmic optimization can be applied to large scale infrastructure within the United States. By utilizing the METAL graph data, we were able to model a complex transportation network and address the problem of maximizing geographical railway coverage while also conserving as many resources as possible.

Dijkstra's Algorithm provided the necessary foundation for spatial analysis, allowing us to accurately determine distances between nodes and identify "coverage gaps" within the existing network. The Greedy Set Cover approach proved to be a highly effective method for managing the NP-hard nature of the rail station location problem. While finding a mathematically perfect minimum is computationally exhaustive, the greedy method yielded a practical and efficient solution that significantly reduced the number of additional stations required to meet our coverage goals.

Our results suggest that strategic placement is the key to expanding railway accessibility, rather than sheer volume. By identifying multiple high impact points we are able to move closer to a national network where rail travel is possible for all. And while there might be many considerations needed to take for real implementations , this project still shows that it is feasible question to be asked. Future versions of this study could consider population density or terrain-based cost analysis to create an even more efficient rail system.